

3

Diving into Variables, Types, and Methods

The initial steps into any programming language are plagued with a fundamental issue—you can understand the words being typed out, but not the meaning behind them. Normally, this would be cause for a paradox, but programming is a special case.

C# is not its own language; it's written in English. The discrepancy between the words you use every day and the code in Visual Studio comes from missing context, which is something that must be learned all over again. You know how to say and spell the words used in C#, but what you don't know is where, when, why, and, most importantly, how they make up the syntax of the language.

This chapter marks our departure from programming theory and the beginning of our journey into actual coding. We'll talk about accepted formatting, debugging techniques, and putting together more complex examples of variables and methods. There's a lot of ground to cover, but by the time you reach the last quiz, you'll be comfortable with the following high-level topics:

- Writing proper C#
- Debugging your code
- Understanding variables
- Introducing operators
- Defining methods

Let's get started!

Writing proper C#

Lines of code function like sentences, meaning they need to have some sort of separating or ending character. Every line of C#, called a statement, *must* end with a semicolon to separate them for the code compiler to process.

However, there's a catch that you need to be aware of. Unlike the written word we're all familiar with, a C# statement doesn't technically have to be on a single line; whitespace and new lines are ignored by the code compiler. For example, a simple variable could be written like this:

```
public string FirstName = "Harrison";
```

Alternatively, it could also be written as follows:

```
public
string
FirstName
=
"Harrison";
```

These two code snippets are both perfectly acceptable to Visual Studio, but the second option is highly discouraged in the software community as it makes code extremely hard to read. The idea is to write your programs as efficiently and clearly as possible.

There will be times when a statement will be too long to reasonably fit on a single line, but those are few and far between. Just make sure that it's formatted in a way someone else could understand, and don't forget the semicolon.

The second formatting rule you need to drill into your coding muscle memory is the use of curly brackets or braces: { }. Methods, classes, and interfaces all need a set of curly brackets after their declaration. We'll talk about each of these in depth later on in the book, but it's important to get the standard formatting in your head early on.

The traditional practice in C# is to include each bracket on a new line, as shown in the following method:

```
public void MethodName()
{
}
```

However, you might see the first curly bracket located on the same line as the declaration out in the wild. It's all down to personal preference:

```
public void MethodName() {  
}
```

While this isn't something to tear your hair out over, the important thing is to be consistent. In this book, we'll stick with "pure" C# code, which will always put each bracket on a new line, while C# examples that have to do with Unity and game development will often follow the second example.

A good, consistent formatting style is paramount when starting in programming, but so is being able to see the fruits of your work. In the next section, we'll talk about how to print out variables and information straight to the Unity console.

Debugging your code

While we're working through practical examples, we'll need a way to print out information and feedback to the **Console** window in the Unity editor. This information is just to help you mark where and when things are happening in your code and won't be visible in the game itself. The programmatic term for this is **debugging**, and both C# and Unity provide helper methods to make this process easier for developers. You already debugged your code from the last chapter, but we didn't go into much detail about how it actually works. Let's fix that.

Whenever I ask you to debug or print something out, use one of the following methods:

- For simple text or individual variables, use the standard `Debug.Log()` method. The text needs to be inside a set of parentheses, and variables can be used directly with no added characters, for example:

```
Debug.Log("Text goes here.");  
Debug.Log(CurrentAge);
```

This will produce the following in the **Console** panel:

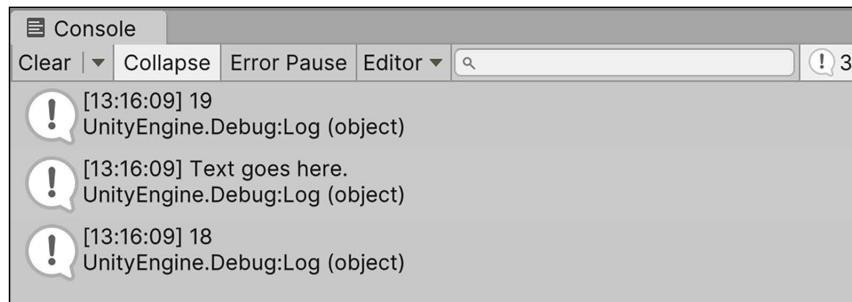


Figure 3.1: Observing `Debug.Log` output

To provide a complete view of the Unity editor, all our screenshots are taken in full-screen mode. For color versions of all book images, use the link below: <https://packt.link/7yy5V>.

- For more complex debugging, use `Debug.LogFormat()`. This will let you place variables inside the printed text by using placeholders. These are marked with a pair of curly brackets, each containing an index. An index is a regular number, starting at 0 and increasing sequentially by 1. In the following example, the `{0}` placeholder is replaced with the `CurrentAge` value, `{1}` with `FirstName`, and so on:

```
Debug.LogFormat("Text goes here, add {0} and {1} as variable  
placeholders", CurrentAge, FirstName);
```

This will produce the following in the **Console** panel:

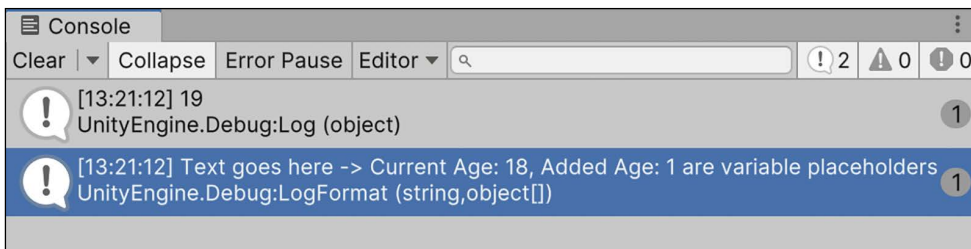


Figure 3.2: Observing `Debug.LogFormat`

You might have noticed that we're using **dot notation** in our debugging techniques, and you'd be right! `Debug` is the class we're using, and `Log()` and `LogFormat()` are different methods that we can use from that class.

With the power of debugging under our belts, we can safely move on and dive into how variables are declared, as well as the different ways that syntax can play out.

Understanding variables

In the previous chapter, we saw how variables are written and touched on the high-level functionality that they provide. However, we're still missing the syntax that makes all that possible, so let's start with the very basics: declaring variables.

Declaring variables

Variables don't just appear at the top of a C# script; they have to be declared according to certain rules and requirements. At its most basic level, a variable statement needs to satisfy the following requirements:

- The type of data the variable will store needs to be specified
- The variable has to have a unique name
- If there is an assigned value, it must match the specified type
- The variable declaration needs to end with a semicolon

The result of adhering to these rules is the following syntax:

```
dataType UniqueName = value;
```



Variables need unique names to avoid conflicts with words that have already been taken by C#, which are called **keywords**. You can find the full list of protected keywords at: <https://docs.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/index>.

This is simple, neat, and efficient. However, a programming language wouldn't be useful in the long run if there was only one way of creating something as pervasive as variables. Complex applications and games have different use cases and scenarios, all of which have unique C# syntax. Let's take a look at these in the next two sections as we explore type and value declarations.

Type and value declarations

The most common scenario for creating variables is one that has all of the required information available when the declaration is made. For instance, if we knew a player's age, storing it would be as easy as doing the following:

```
int CurrentAge = 32;
```

Here, all of the basic requirements have been met:

- A data type is specified, which is `int` (short for integer, which is a fancy word for a whole number)
- A unique name is used, which is `CurrentAge`
- `32` is an integer, which matches the specified data type
- The statement ends with a semicolon

However, there will be scenarios where you'll want to declare a variable without knowing its value right away. We'll talk about this topic in the following section.

Type-only declarations

Consider another scenario—you know the type of data you want a variable to store and its name, but not its value. The value will be computed and assigned somewhere else, but you still need to declare the variable at the top of the script. This situation is perfect for a type-only declaration:

```
int CurrentAge;
```

Only the type (`int`) and unique name (`CurrentAge`) are defined, but the statement is still valid because we've followed the rules. With no assigned value, default values will be assigned according to the variable's type. In this case, `CurrentAge` will be set to `0`, which matches the `int` type. As soon as the actual value of the variable becomes available, it can easily be set in a separate statement by referencing the variable name and assigning it a value:

```
CurrentAge = 32;
```



You can find a complete list of all C# types and their default values at: <https://docs.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/default-values>.

At this point, you might be asking why, so far, our variables haven't included the `public` keyword, called an *access modifier*, which we saw in earlier scripting examples. The answer is that we didn't have the necessary foundation to talk about them with any clarity. Now that we have that foundation, it's time to revisit them in detail.

Using access modifiers

Now that the basic syntax is no longer a mystery, let's get into the finer details of variable statements. Since we read code from left to right, it makes sense to begin our variable deep dive with the keyword that traditionally comes first—an **access modifier**.

Take a quick look back at the variables we used in the preceding chapter in `LearningCurve` and you'll see they had an extra keyword at the front of their statements: `public`. This is the variable's access modifier. Think of it as a security setting, determining who and what can access the variable's information.

Any variable that isn't marked `public` is defaulted to `private` and won't show up in the Unity **Inspector** panel. If you include a modifier, the updated syntax recipe we put together at the beginning of this chapter will look like this:

```
accessModifier dataType UniqueName = value;
```

While explicit access modifiers aren't necessary when declaring a variable, it's a good habit to get into as a new programmer. That extra word goes a long way toward readability and professionalism in your code.

There are four main access modifiers available in C#, but the two you'll be working with most often as a beginner are the following:

- **Public:** This is available to any script without restriction.
- **Private:** This is only available in the class they're created in (which is called the containing class). Any variable without an access modifier will default to Private.

The two advanced modifiers have the following characteristics:

- **Protected:** Accessible from their containing class or types derived from it.
- **Internal:** Only available in the current assembly (an automatically generated file that bundles your code, resources, and pretty much everything else together in a neat package).

There are specific use cases for each of these modifiers, but until we get to the advanced chapters, don't worry about **protected** and **internal**.



Two combined modifiers also exist, but we won't be using them in this book. You can find more information about them at: <https://docs.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/access-modifiers>.

Let's try out some access modifiers of our own! Just like information in real life, some data needs to be protected or shared with specific people. If there's no need for a variable to be changed in the **Inspector** window or accessed from other scripts, it's a good candidate for a private access modifier.

Perform the following steps to update LearningCurve:

1. Change the access modifier in front of CurrentAge from public to private and save the file.
2. Go back into Unity, select the **Main Camera**, and take a look at what changed in the LearningCurve section:

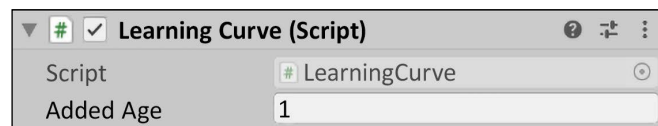


Figure 3.3: LearningCurve script component attached to the Main Camera

Since `CurrentAge` is now `private`, it's no longer visible in the **Inspector** window and can only be accessed within the `LearningCurve` script in code. If we click **Play**, the script will still work exactly as it did before.

This is a good start on our journey into variables, but we still need to know more about what kinds of data they can store. This is where data types come in, which we'll look at in the next section.

Working with types

Assigning a specific type to a variable is an important choice, one that trickles down into every interaction a variable has over its entire lifespan. Since C# is what's called a *strongly typed* or *type-safe* language, every variable must have a data type without exception. In comparison, programming languages like JavaScript, for example, are non-type-safe. This means that there are specific rules when it comes to performing operations with certain types, and regulations when converting a given variable type into another.

Common built-in types

All data types in C# trickle down (or **derive**, in programmatic terms) from a common ancestor: `System.Object`. This hierarchy, called the Common Type System (CTS), means that different types have a lot of shared functionality. The following table lays out some of the most common data type options and the values they store:

Type	Contents of the variable
int	A simple integer, such as the number 3
float	A number with a decimal, such as the number 3.14
string	Characters in double quotes, such as, "Watch me go now"
bool	A Boolean, either true or false

Figure 3.4: Common data types for variables

In addition to specifying the kind of value a variable can store, types contain added information about themselves, including the following:

- Required storage space
- Minimum and maximum values
- Allowed operations
- Location in memory
- Accessible methods
- Base (derived) type

If this seems overwhelming, take a deep breath. Working with all of the types C# offers is a perfect example of using documentation over memorization. Pretty soon, using even the most complex custom types will feel like second nature.



You can find a complete list of all of the C# built-in types and their specifications at: <https://docs.microsoft.com/en-us/dotnet/csharp/programming-guide/types/index>.

Before the list of types in *Figure 3.4* gets too overwhelming, it's best to experiment with them. After all, the best way to learn something new is to use it, break it, and then learn to fix it.

Go ahead and open up *LearningCurve* and add a new variable for each type in the preceding table from the *Common built-in types* section. The names and values you use are up to you; just make sure they're marked as public so we can see them in the **Inspector** window. If you need inspiration, take a look at my code:

```
public class LearningCurve : MonoBehaviour
{
    private int CurrentAge = 30;
    public int AddedAge = 1;

    public float Pi = 3.14f;
    public string FirstName = "Harrison";
    public bool IsAuthor = true;

    // Start is called before the first frame update
    void Start()
    {
        ComputeAge();
    }

    /// <summary>
    /// Time for action - adding comments
    /// Computes a modified age integer
    /// </summary>
    void ComputeAge()
```

```

    {
        Debug.Log(CurrentAge + AddedAge);
    }
}

```

When dealing with string types, the actual text value needs to be inside a pair of double quotes, while float values need to end with a lowercase `f`, as you can see with `FirstName` and `Pi`.

All our different variable types are now visible. Take note of the `bool` variable that Unity displays as a checkbox (true is checked and false is unchecked).

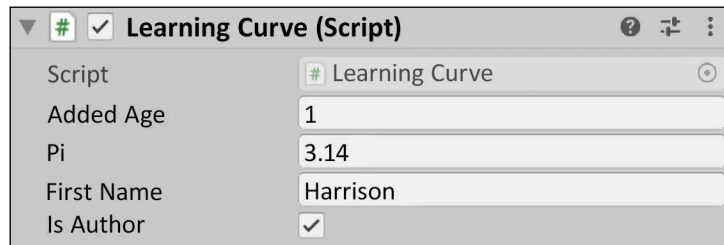


Figure 3.5: *LearningCurve* script component with common variable types

Remember, any variables you declare as `private` won't show up in the **Inspector** window. Before we move on to conversions, we need to touch on a common and powerful application of the **string data type**—namely, the creation of strings that have variables interspersed at will.

While number types behave as you'd expect from grade school math, strings are a different story. It's possible to insert variables and literal values directly into text by starting with a `$` character, which is called **string interpolation**. You've already used an interpolated string in your `LogFormat()` debugging; adding the `$` character lets you use them anywhere!

Let's create a simple interpolated string of our own inside `LearningCurve` to see this in action. Print out the interpolated string inside the `Start()` method directly after `ComputeAge()` is called:

```

void Start()
{
    ComputeAge();
    Debug.Log($"A string can have variables like {FirstName} inserted
    directly!");
}

```

In the console, this string will output the following text:

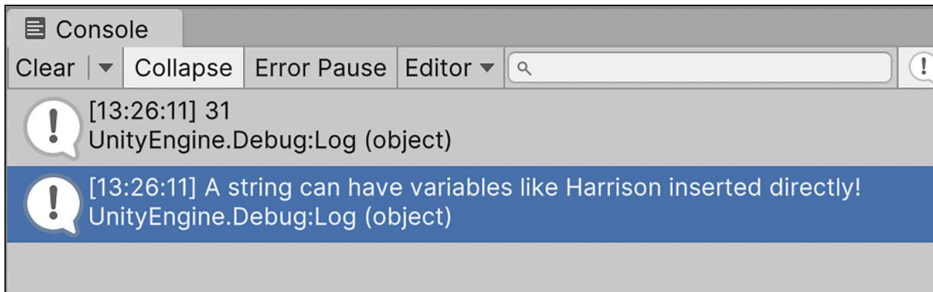


Figure 3.6: Console showing debug log output

Notice that because **Current Age** is now private, it defaults to using the value we set in code (30). It's also possible to create interpolated strings using the `+` operator, which we'll talk about in the *Introducing operators* section. For now, let's move on to type conversions.

Type conversions

We've already seen that variables can only hold values of their declared types, but there will be situations where you'll need to combine variables of different types. In programming terminology, these are called conversions, and they come in two main flavors:

- **Implicit** conversions take place automatically, usually when a smaller value will fit into another variable type without any rounding. For example, any integer can be implicitly converted into a double or float value without additional code:

```
int MyInteger = 3;
float MyFloat = MyInteger;

Debug.Log(MyInteger);
Debug.Log(MyFloat);
```

The output in the **Console** panel can be seen in the following screenshot, where you can see both numbers showing **3** (even though the float is technically 3.0, the zero is rounded off in C#):

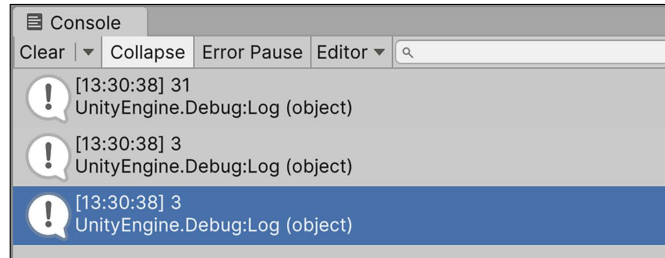


Figure 3.7: Implicit type conversion debug log output

- **Explicit** conversions are needed when there is a risk of losing a variable's information during the conversion. For example, if we wanted to convert a double value into an int value, we would have to explicitly cast (convert) it by adding the destination type in parentheses before the value we want to convert.

This tells the compiler that we are aware that data (or precision) might be lost:

```
int ExplicitConversion = (int)3.14;
Debug.Log(ExplicitConversion);
```

In this explicit conversion, 3.14 would be rounded down to 3, losing the decimal values:

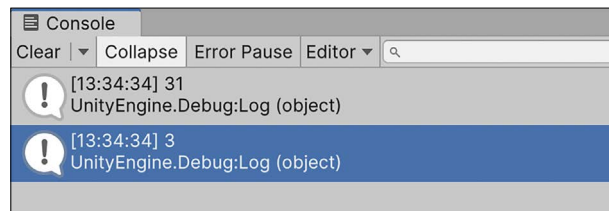


Figure 3.8: Explicit type conversion debug log output



C# provides built-in methods for explicitly converting values to common types. For example, any type can be converted into a string value with the `ToString()` method, while the `Convert` class can handle more complicated conversions. You can find more info about these features under the *Methods* section at: <https://docs.microsoft.com/en-us/dotnet/api/system.convert?view=netframework-4.7.2>.

So far, we've learned that types have rules regarding their interactions, operations, and conversion, but how do we handle a situation where we need to store a variable of an unknown type? This might sound crazy, but think about a data-download scenario—you know the information is coming into your game, but you're not sure what form it will take. We'll discuss how to handle this in the following section.

Inferred declarations

Luckily, C# can *infer* a variable's type from its assigned value. For example, the `var` keyword can let the program know that the type of the data, `CurrentAge`, needs to be determined by its value of 32, which is an integer:

```
var CurrentAge = 32;
```

While this is handy in certain situations, don't be suckered into the lazy programming habit of using inferred variable declarations for everything. This adds a lot of guesswork to your code, where it should be crystal clear. Inferred variable declarations should really only be used when you're testing code and don't know the data type that's being stored. Once you do, changing the variable declaration to the specific type is recommended to avoid runtime errors later on. Before we wrap up our discussion on data types and conversion, we do need to briefly touch on the idea of creating custom types, which we'll do next.

Custom types

When we're talking about data types, it's important to understand early on that numbers and words (referred to as *literal values*) are not the only kinds of values a variable can store. For instance, a class, struct, or enumeration can be stored as variables. We will introduce these topics in *Chapter 5, Working with Classes, Structs, and OOP*, and explore them in greater detail in *Chapter 10, Revisiting Types, Methods, and Classes*.

Types are complicated, and the only way to get comfortable with them is by using them. However, here are some important things to keep in mind:

- All variables need to have a specified type (be it explicit or inferred)
- Variables can only hold values of their assigned type (a string value can't be assigned an int variable)
- If a variable needs to be assigned or combined with a variable of a different type, a conversion needs to take place (either implicit or explicit)
- The C# compiler can infer a variable's type from its value using the `var` keyword, but should only be used when the type isn't known when it's created

That's a lot of nitty-gritty detail we've just jammed into a few sections, but we're not done yet. We still need to understand how naming conventions work in C#, as well as where the variables live in our scripts.

Naming variables

Picking names for your variables might seem like an afterthought in light of everything we've learned about access modifiers and types, but it shouldn't be a straightforward choice. Clear and consistent naming conventions in your code will not only make it more readable but will also ensure that other developers on your team understand your intentions without having to ask.

The first rule when it comes to naming a variable is that the name you give it should be meaningful; the second rule is that you use Pascal case (where the first letter of each compound word in a variable name is capitalized — i.e., HelloWorld).



C# does support camel casing (where the first letter of the variable name doesn't need to be capitalized — i.e., helloWorld), so if you're coming from another programming language like Java or C and are more comfortable with that formatting, the compiler won't complain. Pascal case is the more generally accepted way of doing things in C#, so we'll use that going forward.

Let's take a common example from games and declare a variable to store a player's health:

```
public int Health = 100;
```

If you find yourself declaring a variable like this, alarm bells should be going off in your head. Whose health? Is it storing the maximum or minimum value? What other code will be affected when this value changes? These are all questions that should be easily answered by a meaningful variable name; you don't want to find yourself confused by your own code in a week or a month.

With that said, let's try to make this a bit better using a Pascal case name:

```
public int MaxPlayerHealth = 100;
```

Remember, Pascal case starts each word in the variable name with an uppercase letter.

That's much better. With a little thought, we've updated the variable name with meaning and context. Since there is no technical limit in terms of how long a variable name can be, you might find yourself going overboard and writing out ridiculously descriptive names, which will give you problems just as much as a short, non-descriptive name would.

As a general rule, make a variable name as descriptive as it needs to be—no more, no less. Find your style and stick to it.

Understanding variable scope

We're getting to the end of our dive into variables, but there's still one more important topic we need to cover: **scope**. Similar to access modifiers, which determine the outside classes that can grab a variable's information, the variable **scope** is the term used to describe where a given variable exists and its access point within its containing class.

There are three main levels of variable scope in C#:

- **Global scope** refers to a variable that can be accessed by an entire program; in this case, a game. C# doesn't directly support global variables, but the concept is useful in certain cases, which we'll cover in *Chapter 10, Revisiting Types, Methods, and Classes*.
- **Class scope** or **member scope** refers to a variable that is accessible anywhere in its containing class.
- **Local scope** refers to a variable that is only accessible inside the specific method or block of code it's created in.

Take a look at the following screenshot. You don't need to put this into LearningCurve if you don't want to; it's only for visualization purposes at this point:

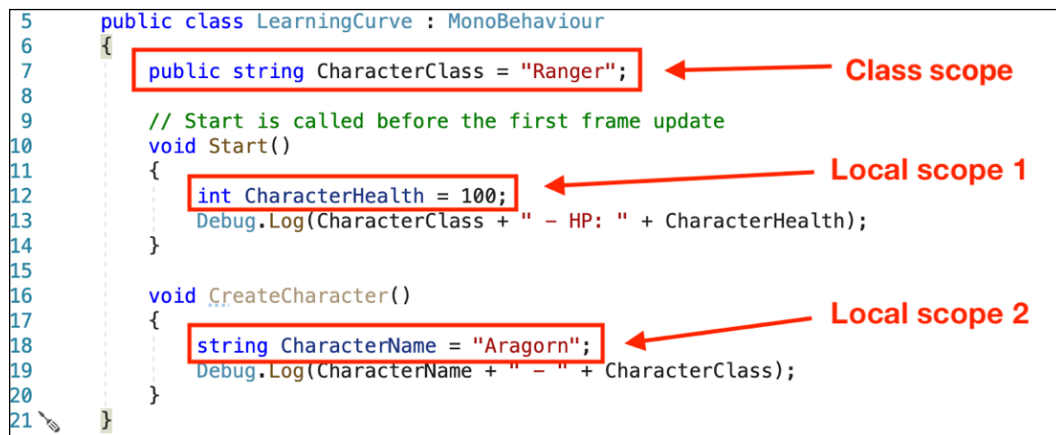


Figure 3.9: Diagram of different scopes in the LearningCurve script

When we talk about code blocks, we're referring to the area inside any set of curly brackets. These brackets serve as a kind of visual hierarchy in programming; the farther right-indented they are, the deeper they are nested in the class.

Let's break down the class and local scope variables in *Figure 3.9*:

- `CharacterClass` is declared at the very top of the class, which means we can reference it by name anywhere inside `LearningCurve`. You might hear this concept referred to as **variable visibility**, which is a good way of thinking about it.
- `CharacterHealth` is declared inside the `Start()` method, which means it is only visible inside that block of code. We can still access `CharacterClass` from `Start()` with no issue, but if we attempted to access `CharacterHealth` from anywhere but `Start()`, we would receive an error.
- `CharacterName` is in the same boat as `CharacterHealth`; it can only be accessed from the `CreateCharacter()` method. This was just to illustrate that there can be multiple, even nested, local scopes in a single class.

If you spend enough time around programmers, you'll hear discussions (or arguments, depending on the time of day) about the best place to declare a variable. The answer is simpler than you might think: variables should be declared with their use in mind. If you have a variable that needs to be accessed throughout a class, make it a class variable. If you only need a variable in a specific section of code, declare it as a local variable.



Note that only class variables can be viewed in the **Inspector** window, which isn't an option for local or global variables.

With naming and scope in our toolbox, let's transport ourselves back to middle school math class and relearn how arithmetic operations work all over again!

Introducing operators

Operator symbols in programming languages represent the *arithmetic*, *assignment*, *relational*, and *logical* functionality that types can perform. Arithmetic operators represent basic math functions, while assignment operators perform math and assignment functions together on a given value. Relational and logical operators evaluate conditions between multiple values, such as *greater than*, *less than*, and *equal to*.

C# also offers bitwise and miscellaneous operators, but these won't come into play for you until you're well on your way to creating more complex applications. At this point, it only makes sense to cover arithmetic and assignment operators; we'll get to relational and logical functionality when it becomes relevant in *Chapter 4, Control Flow and Collection Types*.

Arithmetic and assignments

You're already familiar with the arithmetic operator symbols from school:

- + for addition
- - for subtraction
- / for division
- * for multiplication

C# operators follow the conventional order of operations, that is, evaluating parentheses first, then exponents, then multiplication, then division, then addition, and finally, subtraction.

For instance, the following equations will provide different results, even though they contain the same values and operators:

```
5 + 4 - 3 / 2 * 1 = 8
5 + (4 - 3) / 2 * 1 = 5
```

Operators work the same when applied to variables as they do with literal values. Assignment operators can be used as a shorthand replacement for any math operation by using any arithmetic and equals symbol together. For example, if we wanted to multiply a variable, you could use the following code:

```
int CurrentAge = 32;
CurrentAge = CurrentAge * 2;
```

The second, alternative, way to do this is shown here:

```
int CurrentAge = 32;
CurrentAge *= 2;
```

The equals symbol is also considered an assignment operator in C#. The other assignment symbols follow the same syntax pattern as our preceding multiplication example: +=, -=, and /= for add and assign, subtract and assign, and divide and assign, respectively.

Strings are a special case when it comes to operators as they can use the addition symbol to create patchwork text. For example, if you put the following code into the Start method:

```
string FullName = "Harrison " + "Ferrone";
```

This will produce the following when logged in to the **Console** panel:

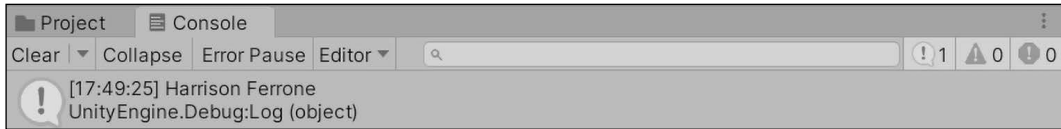


Figure 3.10: Using operators on strings

This approach tends to produce clunky code, making string interpolation the preferred method for putting together different bits of text in most cases.

Take note that arithmetic operators don't work on all data types. For example, the `*` and `/` operators don't work on string values, and none of these operators work on Booleans. Having learned that types have rules that govern what kind of operations and interactions they can have, we'll give it a shot in the *Defining methods* section.

Let's do a little experiment: we'll try to multiply our string and float variables together, as we did earlier with our numbers:

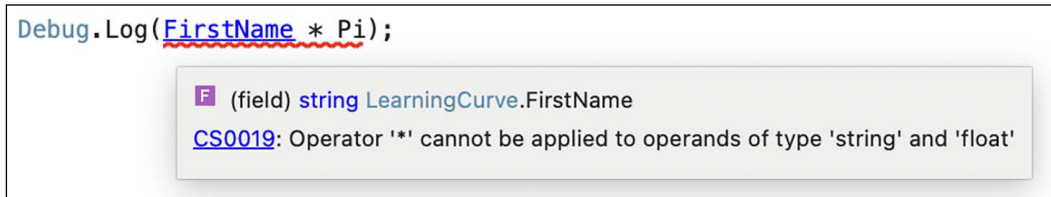


Figure 3.11: Visual Studio incorrect type operation error message

Look at Visual Studio and you'll see we've got an error message letting us know that a string type and a float type can't be multiplied. This error will also show up in the Unity **Console**, and won't let the project build.

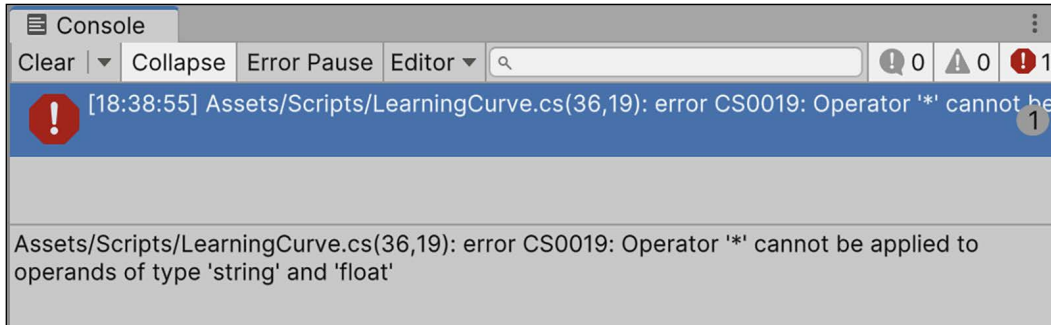


Figure 3.12: Console showing operator errors on incompatible data types

Whenever you see this type of error, go back and inspect your variable types for incompatibilities.

We must clean up this example, as the compiler won't allow us to run our game at this point. Choose between a pair of backslashes (//) at the beginning of the line `Debug.Log(FirstName*Pi)`, or delete it altogether.

That's as far as we need to go in terms of variables and types for the moment. Be sure to test yourself on the quiz at the end of this chapter before moving on!

Defining methods

In the previous chapter, we briefly touched on the role methods play in our programs; namely, that they store and execute instructions, just like variables store values. Now, we need to understand the syntax of method declarations and how they drive action and behavior in our classes.

As with variables, method declarations have their basic requirements, which are as follows:

- The type of data that will be returned by the method (methods don't all have to return anything, so this can be void)
- A unique name, starting with a capital letter
- A pair of parentheses following the method name
- A pair of curly brackets marking the method body (where instructions are stored)

Putting all of these rules together, we get a simple method blueprint:

```
returnType UniqueName()  
{  
    method body  
}
```

Let's break down the default `Start()` method in `LearningCurve` as a practical example:

```
void Start()  
{  
}
```

In the preceding output, we can see the following:

- The method starts with the `void` keyword, which is used as the method's return type if it doesn't return anything.
- The method has a unique name within the class. You can use the same name in different classes, but you should aim to always make your names unique no matter what.

- The method has a pair of parentheses after its name to hold any potential parameters.
- The method body is defined by a set of curly brackets.

In general, if you have a method that has an empty method body, it's good practice to delete it from the class. You always want to be pruning your scripts of unused code.

Like variables, methods can also have security levels. However, they can also have input parameters, both of which we'll be discussing next!

Declaring methods

Methods can also have the same four access modifiers that are available to variables, as well as input parameters. Parameters are variable placeholders that can be passed into methods and accessed inside them. The number of input parameters you can use isn't limited, but each one needs to be separated by a comma, show its data type, and have a unique name.

Think of method parameters as variable placeholders whose values can be used inside the method body. If we apply these options, our updated blueprint will look like this:

```
accessModifier returnType UniqueName(parameterType parameterName)
{
    method body
}
```

If there is no explicit access modifier, the method defaults to private. A private method, like a private variable, cannot be called from other scripts. To call a method (meaning to run or execute its instructions), we simply use its name, followed by a pair of parentheses, with or without parameters, and cap it off with a semicolon:

```
// Without parameters
UniqueName();
// With parameters
UniqueName(parameterVariable);
```

Like variables, every method has a fingerprint that describes its access level, return type, and parameters. This is called its method signature. Essentially, a method's signature marks it as unique to the compiler so Visual Studio knows what to do with it.

Now that we understand how methods are structured, let's create one of our own.

The *Methods are placeholders too* section in the previous chapter had you blindly copy a method called `ComputeAge()` into `LearningCurve` without you knowing what you were getting into. This time, let's purposefully create a method:

1. Declare a public method with a void return type called `GenerateCharacter()`:

```
public void GenerateCharacter()  
{  
}
```

2. Add a simple `Debug.Log()` inside the new method and print out a character name from your favorite game or movie:

```
Debug.Log("Character: Spike");
```

3. Call `GenerateCharacter()` inside the `Start()` method:

```
void Start()  
{  
    GenerateCharacter();  
}
```

4. Hit **Play** to see the output.

When the game starts up, Unity automatically calls `Start()`, which, in turn, calls our `GenerateCharacter()` method and prints the result to the **Console** window.

If you have read enough documentation, you'll see different terminology related to methods. Throughout the rest of this book, when a method is created or declared, I'll refer to this as defining a method. Similarly, I'll refer to running or executing a method as calling that method.

The power of naming is integral to the entirety of the programming landscape, so it shouldn't be a surprise that we're going to revisit naming conventions for methods before moving on.

Naming conventions

Like variables, methods need unique, meaningful names to distinguish them in code. Methods drive actions, so it's a good practice to name them with that in mind. For example, `GenerateCharacter()` sounds like a command, which reads well when you call it in a script, whereas a name such as `Summary()` is bland and doesn't paint a very clear picture of what the method will accomplish. Like variables, method names are written in Pascal case.

Methods as logic detours

We've seen that lines of code execute sequentially in the order they're written, but bringing methods into the picture introduces a unique situation. Calling a method tells the program to take a detour into the method instructions, run them one by one, and then resume sequential execution where the method was called.

Take a look at the following screenshot and see whether you can figure out in what order the debug logs will be printed out to the console:

```
13 // Use this for initialization
14 void Start ()
15 {
16     Debug.Log("Choose" a character.");
17     GenerateCharacter();
18     Debug.Log("A fine choice.");
19 }
20
21 public void GenerateCharacter()
22 {
23     Debug.Log("Character: Spike");
24 }
```

Figure 3.13: Considering the order of debug logs

These are the steps that occur:

1. Choose a character prints out first because it's the first line of code.
2. When GenerateCharacter() is called, the program jumps to line 23, prints out Character: Spike, and then resumes execution at line 17.
3. A fine choice prints out last, after all the lines in GenerateCharacter() have finished running.

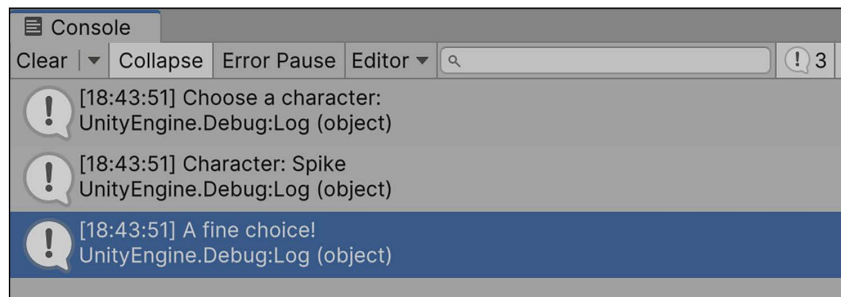


Figure 3.14: Console showing the output of character-building code

Now, methods in themselves wouldn't be very useful beyond simple examples like these if we couldn't add parameter values to them, which is what we'll do next.

Specifying parameters

Chances are your methods aren't always going to be as simple as `GenerateCharacter()`. To pass in additional information, we'll need to define parameters that our method can accept and work with. Every method parameter is an instruction and needs to have two things:

- An explicit type
- A unique name

Does this sound familiar? Method parameters are essentially stripped-down variable declarations and perform the same function. Each parameter acts like a local variable, only accessible inside their specific method.

You can have as many parameters as you need. Whether you're writing custom methods or using built-in ones, the parameters that are defined are what the method requires to perform its specified task.

If parameters are the blueprint for the types of values a method can accept, then arguments are the values themselves. To break this down further, consider the following:

- The argument that's passed into a method needs to match the parameter type, just like a variable type and its value
- Arguments can be literal values (for instance, the number 2) or other variables you've declared in the class



Argument names and parameter names don't need to match to compile.

Now, let's move on and add some method parameters to make `GenerateCharacter()` a bit more interesting.

Let's update `GenerateCharacter()` so that it can take in two parameters:

1. Add two method parameters: one for a character's name of the `string` type, and another for a character's level of the `int` type:

```
public void GenerateCharacter(string name, int level)
{
}
```

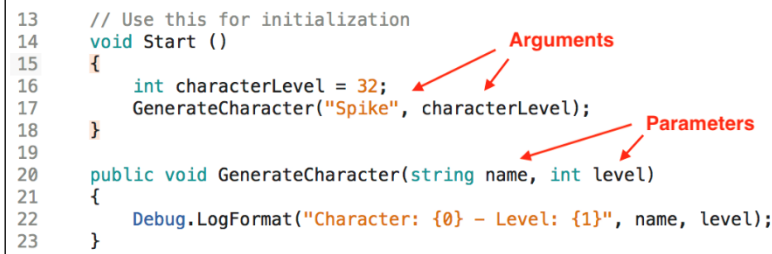
2. Update `Debug.Log()` so that it uses these new parameters:

```
Debug.LogFormat("Character: {0} - Level: {1}", name, level);
```

3. Update the `GenerateCharacter()` method call in `Start()` with your arguments, which can be either literal values or declared variables:

```
int CharacterLevel = 32;
GenerateCharacter("Spike", CharacterLevel);
```

Your code should look like the following:



```
13 // Use this for initialization
14 void Start ()
15 {
16     int characterLevel = 32;
17     GenerateCharacter("Spike", characterLevel);
18 }
19
20 public void GenerateCharacter(string name, int level)
21 {
22     Debug.LogFormat("Character: {0} - Level: {1}", name, level);
23 }
```

Figure 3.15: Updating the `GenerateCharacter()` method

Here, we defined two parameters, `name` (string) and `level` (int), and used them inside the `GenerateCharacter()` method, just like local variables. When we called the method inside `Start()`, we added argument values for each parameter with corresponding types. In Figure 3.15, you can see that using the literal string value in quotations produced the same result as using `characterLevel`.

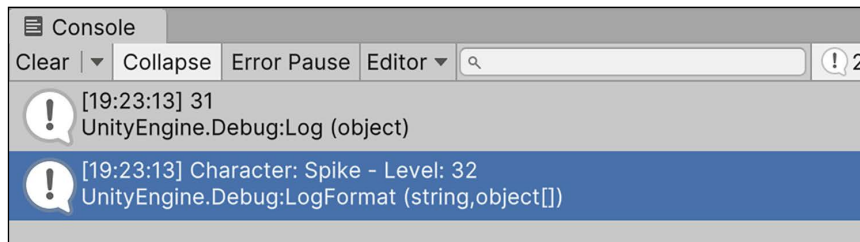


Figure 3.16: Console showing the output from method parameters

Going even further with methods, you might be wondering how we can pass values from inside the method and back out again. This brings us to our next section on return values.

Specifying return values

Aside from accepting parameters, methods can return values of any C# type. All of our previous examples have used the void type, which doesn't return anything, but being able to write instructions and pass back computed results is where methods shine.

According to our blueprints, method return types are specified after the access modifier. In addition to the type, the method needs to contain the return keyword, followed by the return value. A return value can be a variable, a literal value, or even an expression, as long as it matches the declared return type.

Methods that have a return type of void can still use the return keyword with no value or expression assigned.

Once the line with the return keyword is reached, the method will stop executing. This is useful in cases where you want to check whether a value or values exist before continuing or guard against program crashes.

Next, add a return type to `GenerateCharacter()` and learn how to capture it in a variable. Let's update the `GenerateCharacter()` method so that it returns an integer:

1. Change the return type in the method declaration from void to int, and set the return value to `level += 5` using the return keyword:

```
public int GenerateCharacter(string name, int level)
{
    Debug.LogFormat("Character: {0} - Level: {1}", name, level);

    return level += 5;
}
```

2. `GenerateCharacter()` will now return an integer. This is computed by adding 5 to the level argument. We haven't specified how, or if, we want to use this return value, which means that right now, the script won't do anything new.

Now, the question becomes: how do we capture and use the newly added return value? Well, we'll discuss that very topic in the following section.

Using return values

When it comes to using return values, there are two approaches available:

- Create a local variable to capture (store) the returned value.
- Use the calling method itself as a stand-in for the returned value, using it just like a variable. The calling method is the actual line of code that fires the instructions, which, in our example, would be `GenerateCharacter("Spike", CharacterLevel)`. You can even pass a calling method into another method as an argument if need be.

The first option is preferred in most programming circles for its readability. Throwing around method calls as variables can get messy fast, especially when we use them as arguments in other methods.

Let's give this a try in our code by capturing and debugging the return value that `GenerateCharacter()` returns.

We're going to use both ways of capturing and using return variables with two simple debug logs:

1. Create a new `int` variable in the `Start` method called `nextSkillLevel` and assign it to the return value of the `GenerateCharacter()` method call we already have in place:

```
int nextSkillLevel = GenerateCharacter("Spike", characterLevel);
```

2. Add two debug logs, with the first printing out `nextSkillLevel` and the second printing out a new calling method with argument values of your choice:

```
Debug.Log(nextSkillLevel);  
Debug.Log(GenerateCharacter("Faye", characterLevel));
```

3. Comment out the debug log inside `GenerateCharacter()` with two forward slashes (`//`) to make the console output less cluttered. Your code should look like the following:

```
// Start is called before the first frame update  
void Start()  
{  
    int characterLevel = 32;  
    int nextSkillLevel = GenerateCharacter("Spike", characterLevel);  
    Debug.Log(nextSkillLevel);  
    Debug.Log(GenerateCharacter("Faye", characterLevel));  
}
```

```
public int GenerateCharacter(string name, int level)
{
    // Debug.LogFormat("Character: {0} - Level: {1}", name, level);
    return level += 5;
}
```

4. Save the file and hit **Play** in Unity. To the compiler, the `nextSkillLevel` variable and the `GenerateCharacter()` method caller represent the same information, namely an integer, which is why both logs show the number 37:

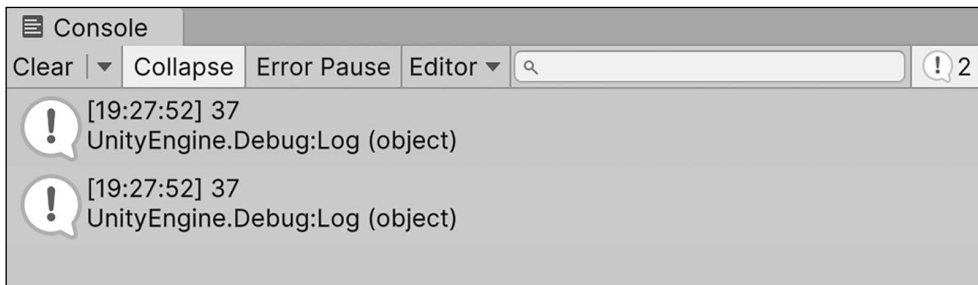


Figure 3.17: Console output from the character generation code

That was a lot to take in, especially given the exponential possibilities of methods with parameters and return values. However, we'll ease off the throttle here for a minute and consider some of Unity's most common methods to catch a little breathing room.

But first, see if you can handle a challenge in the next *Hero's trial*!

Hero's trial—methods as arguments

If you're feeling brave, why not try creating a new method that takes in an `int` parameter and simply prints it out to the console? No return type is necessary. When you've got that, call the method in `Start`, pass in a `GenerateCharacter` method call as its argument, and take a look at the output.

Dissecting common Unity methods

We're now at a point where we can realistically discuss the most common default methods that come with any new Unity C# script: `Start()` and `Update()`. Unlike the methods we define ourselves, methods belonging to the `MonoBehaviour` class are called automatically by the Unity engine according to their respective rules. In most cases, it's important to have at least one `MonoBehaviour` method in a script to kick off your code.



You can find a complete list of all available `MonoBehaviour` methods and their descriptions at: <https://docs.unity3d.com/ScriptReference/MonoBehaviour.html>. You can also find the order in which each method is executed at: <https://docs.unity3d.com/Manual/ExecutionOrder.html>.

Just like stories, it's always a good idea to start at the beginning. So, naturally, we should take a look at every Unity script's first default method—`Start()`.

The `Start()` method

Unity calls the `Start()` method on the first frame where a script is enabled for the first time. Since `MonoBehaviour` scripts are almost always attached to *GameObjects* in a scene, their attached scripts are enabled at the same time they are loaded when you hit play. In our project, `LearningCurve` is attached to the **Main Camera** *GameObject*, which means that its `Start()` method runs when the camera is loaded into the scene. `Start()` is primarily used to set up variables or perform logic that needs to happen before `Update()` runs for the first time.

The examples we've worked on so far have all used `Start()`, even though they weren't performing setup actions, which isn't normally the way it would be used. However, it only fires once, making it an excellent tool to use for displaying one-time-only information on the console.

Other than `Start()`, there's one other major Unity method that you'll run into by default: `Update()`. Let's familiarize ourselves with how it works in the following section before we finish this chapter.

The `Update()` method

If you spend enough time looking at the sample code in the Unity Scripting Reference (<https://docs.unity3d.com/ScriptReference/>), you'll notice that a vast majority of the code is executed using the `Update()` method. As your game runs, the **Scene** window is displayed many times per second, which is called the **frame rate** or **frames per second (FPS)**.

After each frame is displayed, the `Update()` method is called by Unity, making it one of the most executed methods in your game. This makes it ideal for detecting mouse and keyboard input or running gameplay logic.

If you're curious about the FPS rating on your machine, hit play in Unity and click the **Stats** tab in the upper-right corner of the **Game** view:

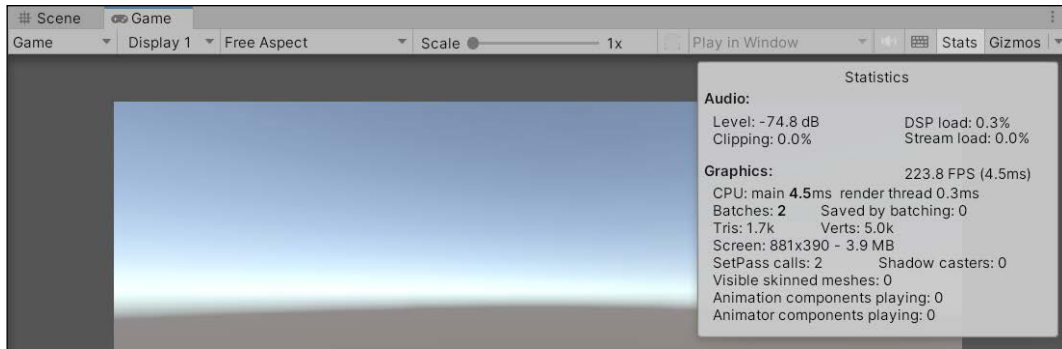


Figure 3.18: Unity editor showing the Stats panel with graphics FPS count

You'll be using the `Start()` and `Update()` methods in the lion's share of your initial C# scripts, so get acquainted with them. That being said, you've reached the end of this chapter with a pocketful of the most fundamental building blocks programming with C# has to offer.

Summary

This chapter has been a fast descent from the basic theory of programming and its building blocks into the strata of real code and C# syntax. We've seen good and bad forms of code formatting, learned how to debug information in the Unity console, and created our first variables.

C# types, access modifiers, and variable scope weren't far behind, as we worked with member variables in the **Inspector** window and started venturing into the realm of methods and actions.

Methods helped us to understand written instructions in code, but more importantly, how to properly harness their power into useful behaviors. Input parameters, return types, and method signatures are all important topics, but the real gift they offer is the potential for new kinds of actions to be performed.

You're now armed with the two fundamental building blocks of programming—variables and methods; almost everything you'll do from now on will be an extension or application of these two concepts.

In the next chapter, we'll take a look at a special subset of C# types called collections, which can store groups of related data, and learn how to write decision-based code.

Pop quiz—variables and methods

1. What is the proper way to write a variable name in C#?
2. How do you make a variable appear in Unity's Inspector window?
3. What are the four access modifiers available in C#?
4. When are explicit conversions needed between types?
5. What are the minimum requirements for defining a method?
6. What is the purpose of the parentheses at the end of the method name?
7. What does a return type of `void` mean in a method definition?
8. How often is the `Update()` method called by Unity?

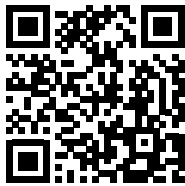
Don't forget to check your answers against mine in the *Pop Quiz Answers* appendix to see how you did!

Join us on discord!

Read this book alongside other users, Unity game development experts and the author himself.

Ask questions, provide solutions to other readers, chat with the author via. Ask Me Anything sessions and much more.

Scan the QR code or visit the link to join the community.



<https://packt.link/csharpwithunity>